

AIを使った開発の知見共有

防災アプリの制作を題材に

開発メンバー勉強会 / 課題1

Kyoko Takazawa

この発表について

- テーマは、**AIを使った開発で得た知見の共有**
- 作った防災アプリは、その **題材**（成果物の紹介が目的ではない）
- 進め方
 - 題材の紹介 → 開発の進め方 → 知見①～④ → まとめ

題材：作ったアプリ

- 災害時に、避難所と避難ルートを提案する Web アプリ
- 災害種別で提案が変わる／危険区域を避けるルート／オフライン対応
- 課題の必須要件（HTTPS・OIDC認証・Git管理）も充足
- 公開URL：<https://kyokoron.github.io/team-challenge/>

数時間で制作。以降は、この制作を通して得た知見を共有する。

開発の進め方

- AI（Claude Code）と 対話しながら 開発
- コード作成は AI、要件定義・レビュー・判断は人 が担当
- 環境構築は不要。ブラウザ版の VS Code で作業し、GitHub の静的サイトとして公開
- 個人でも、サーバー・費用なしで開発から公開まで完結できる

知見① AIの出力は「検証前提」で使う

- AI は、もっともらしい回答を即座に出す。ただし誤りも混ざる
- 例：「DB が必要なのは投稿・更新のときだけ」と断定
→ 問い直すと、オフライン対応・更新頻度・コストなど論点は多かった
- 重要な判断ほど「本当にそうか」と問い返す。鵜呑みにしない

知見②「動く」と「正しい」は別物

動作はするが、誤った挙動が複数あった。

- 内陸で「津波」を選ぶと、海側へ誘導していた
- データ取得に失敗すると、実在しない避難所を表示していた
- 広域データで距離計算が狂い、最寄り順が崩れていた
- 直線距離を、そのまま所要時間として表示していた

一見きちんと動いて見える。動作確認だけでは危うく、要件・安全面のレビューが要る。

知見③ データ整備がボトルネックになりやすい

- コード生成は速い。一方で、データを「使える形」にする作業は残る
- オープンデータ特有の課題
 - どれが正解のデータか分かりにくい
 - ファイルが単位ごとに分割されている（例：洪水想定は河川単位）
 - 古い形式（Shapefile）で、変換・整形が必要

データ整備の工数を最初から見込む。整形自体は AI にも任せられる。

知見④ 人の役割は「実装」から「判断」へ

これまで

ユーザー

↓ UIを操作

UI (アプリ)

↓

機能 (ロジック)

↓

データ

価値の中心：UI設計・機能開発・実装

これから

ユーザー

↓ 目的を伝える

AI (理解・推論)

↓

データ

価値の中心：ドメイン理解・データ整備・課題設定

実装の速さは AI に任せられる。人が担うのは、ドメイン理解・課題設定・レビュー。

まとめ：AI活用の勘所

- AIにより、開発の速度は大きく向上する
- ただし出力は **検証前提**。品質・安全・最終判断は人が担う
- データ整備・課題設定など、**人が担う工程の比重が増す**
- 「作る」こと以上に、「何を・正しく作るか」を決める力が重要になる

付録：技術・データ

主な機能

- 災害種別に応じた避難所ランキング
- 浸水域を避ける避難ルート
- 現在地が危険なときの警告
- オフライン（PWA）／OIDC認証 + HTTPS

技術・データ

- Vanilla JS / MapLibre GL JS / Auth0(OIDC)
- Service Worker + IndexedDB
- 国土地理院（地図・避難所・標高）
- 国土数値情報（洪水浸水想定 A31）
- OpenRouteService（回避ルート）